

Cahier de TP

« Quarkus Reactive »

Pré-requis :

- Git
- JDK21+
- Apache JMeter
- Docker
- kind
- IDE : Eclipse, VSCode ou IntelliJ (Version Ultimate de préférence)

Solutions des ateliers :

<https://github.com/dthibau/quarkus-solutions>

Table des matières

Ateliers 1 : Développer avec Quarkus.....	3
1.1 Démarrage projet.....	3
1.1.1 Création de projet.....	3
1.1.2 Mise en place IDE.....	3
1.2 : Annotations CDI.....	4
1.2.1 Mise à jour pom.xml.....	4
1.2.2 Beans et injection.....	4
1.3 : Configuration.....	4
1.3.1 Interfaces de configuration.....	4
1.3.2 Configuration des traces.....	5
1.4 : Profils.....	5
Ateliers 2 : Programmation réactive.....	6
2.1 Benchmark des modèles de threads.....	6
2.1.1 Mise en place du projet benchmark.....	6
2.1.2 Exécution du benchmark JMeter.....	7
Ateliers 3 : La librairie Mutiny.....	9
3.1 De CompletableFuture à Mutiny.....	9
3.1.1 Mise en place du projet mutiny-playground.....	9
3.1.2 Réécriture en Mutiny.....	9
3.2 Composition d'appels et gestion d'erreurs.....	9
3.2.1 Séquentiel vs parallèle.....	9
3.2.2 Retry, timeout et fallback.....	10
3.3 Concepts avancés.....	12
3.3.1 Effet latéral avec invoke().....	12
3.3.2 Conversion entre Uni et Multi.....	12
3.3.3 Mémoriser avec memoize().....	12
3.4 Tester du code Mutiny.....	13
Ateliers 4 : Couche persistance — Hibernate Reactive et Redis.....	14
4.1 Hibernate Reactive Panache et PostgreSQL.....	14
4.1.1 Mise en place.....	14
4.1.2 Repository Pattern.....	15
4.1.3 Service métier réactif.....	15

4.1.4 Transactions réactives.....	15
4.2 Cache Redis sur les lectures.....	16
4.2.1 Mise en place de Redis.....	16
4.2.2 Lecture avec cache.....	16
Ateliers 5 : Couche web — Quarkus REST.....	17
5.1 Mise en place de endpoints REST.....	17
5.1.1 Ajout les extensions.....	17
5.1.2 Endpoints impératifs et réactifs.....	17
5.1.3 Bean Validation.....	17
5.1.4 ExceptionMapper centralisé.....	18
5.1.5 Tests RestAssured.....	18
Ateliers 6 : Couche service — composition et résilience.....	19
6.1 Quarkus REST Client.....	19
6.1.1 Démarrage des services dépendants.....	19
6.1.2 Déclaration des clients.....	19
6.1.2 Composition en parallèle.....	19
6.2 Politiques de résilience.....	20
6.2.1 Retry et Timeout.....	20
6.2.2 Circuit Breaker et Fallback.....	20
6.2.3 Mise à l'épreuve.....	20
Ateliers 7 : SmallRye Reactive Messaging et Kafka.....	22
7.1 Émission de messages.....	22
7.2 Consommation côté notification-service.....	22
7.3 SSE.....	23

Ateliers 1 : Développer avec Quarkus

1.1 Démarrage projet

Objectifs :

- Installation de Quarkus CLI
- Découverte DevUI
- Création premier projet

1.1.1 Création de projet

Installer Quarkus CLI :

<https://quarkus.io/guides/cli-tooling#installing-the-cli>

Créer un projet **hello-world** avec l'extension rest

```
$ quarkus create app org.formation:hello-world \
--extension=quarkus-rest
$ cd hello-world
$ quarkus dev
```

Accéder à la page d'accueil et :

- La ressource **/hello**
- Visiter la DevUI
- Modifier le code source de *GreetingResource*

Lister les extensions disponibles pour le projet :

```
$ quarkus ext ls
```

1.1.2 Mise en place IDE

Installer les plugins pour l'IDE de votre choix

Puis importer le projet Maven dans votre IDE

Visualiser :

- le fichier **pom.xml**
- la classe présente dans **src/main/java**
- le répertoire **docker**
- le répertoire **src/main/resources**
- la classe de test présente dans **src/test/java**

Ensuite :

1. Visualisez la ressource **/hello** dans un navigateur
2. Modifier le fichier **GreetingResource**
3. Faire un reload dans le navigateur
4. Provoquer une erreur
5. Faire un reload

1.2 : Annotations CDI

Objectifs

- Annotations CDI
- Exemple d'intercepteur

1.2.1 Mise à jour pom.xml

Ajouter une dépendance sur lombok en éditant votre *pom.xml* :

```
<lombok.version>1.18.30</lombok.version>

<dependency>

<groupId>org.projectlombok</groupId>

<artifactId>lombok</artifactId>

<version>${lombok.version}</version>

<scope>provided</scope>

</dependency>
```

Ajouter également une bibliothèque de sérialisation JSON, par exemple :

```
$ quarkus ext add quarkus-rest-jackson
```

1.2.2 Beans et injection

Supprimer le bean *GreetingResource*

Récupérer les sources fournis

Annotez les sources fournis afin que le endpoint **localhost:8080/livraisons** renvoie un tableau d'objets *Livraison* en JSON

1.3 : Configuration

1.3.1 Interfaces de configuration

Configurer Quarkus afin que le serveur écoute sur le port 8000

Définir une interface qui définit les propriétés d'un service de notification offrant une API Restful :

- *protocol* : http ou https, valeur par défaut : « http »
- *host* : Non vide
- *port* : Optionnel
- *rootUrl* : Optionnel
- *token* : Longueur minimale 10 caractères

Ajouter l'extension **quarkus-hibernate-validator**

Ajouter des contraintes de validation et vérifier quelles sont effectives

S'injecter la configuration dans le bean **LivraisonServiceImpl** et afficher les valeurs de config dans la méthode d'initialisation

1.3.2 Configuration des traces

Activer le mode WARN pour le logger root, sauf pour le package *org.formation* qui sera en mode DEBUG

Générer un message de DEBUG dans la classe Controller ou Service

1.4 : Profils

Définir un autre port pour le profil de **prod**

Construire un jar avec

```
./mvnw package
```

Démarrer ensuite l'application via :

```
java -jar target/quarkus-app/quarkus-run.jar
```

Vérifier que le profil **prod** est activé

Définir un autre port pour le profil **staging**

Fixer la propriété *quarkus.application.name* pour ce profil

Construire un jar avec

```
./mvnw package -Dquarkus.profile=staging
```

Démarrer l'application via :

```
java -jar target/quarkus-app/quarkus-run.jar
```

Et vérifier les profils activés

Ateliers 2 : Programmation réactive

2.1 Benchmark des modèles de threads

Objectifs :

- Visualiser la scalabilité comparée de l'impératif bloquant, du réactif et des virtual threads
- Comprendre l'impact d'un blocage I/O simulé sur le pool de threads
- Identifier les situations où l'évent loop offre un avantage tangible

2.1.1 Mise en place du projet benchmark

Créer un projet **benchmark-service** :

```
$ quarkus create app org.formation:benchmark-service \
    --extension=quarkus-rest-jackson
$ cd benchmark-service
$ quarkus dev
```

Créer la ressource **BenchmarkResource** exposant trois endpoints qui simulent chacun une attente I/O de 200 ms sous un modèle d'exécution différent :

```
@Path("/bench")
@Produces(MediaType.APPLICATION_JSON)
public class BenchmarkResource {

    // 1) Impératif bloquant (worker thread)
    @GET @Path("/imperative")
    @Blocking
    public ResponseDto imperative() {
        sleep(200);
        return new ResponseDto("imperative", 200);
    }

    // 2) Réactif (event loop)
    @GET @Path("/reactive")
    public Uni<ResponseDto> reactive() {
        return Uni.createFrom().voidItem()
            .onItem().delayIt().by(Duration.ofMillis(200))
    }
}
```

```

        .onItem().transform(x -> new ResponseDto("reactive",
200));
    }

    // 3) Virtual Threads (Loom)
    @GET @Path("/virtual")
    @RunOnVirtualThread
    public ResponseDto virtualThreads() {
        sleep(200);
        return new ResponseDto("virtual", 200);
    }

    // Bloquage Event-loop
    @GET @Path("/bad-guy")
    @NonBlocking
    public ResponseDto badGuy() {
        sleep(200);
        return new ResponseDto("bad", 200);
    }

    private void sleep(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException
ignored) {}
    }

    public record ResponseDto(String model, long
simulatedLatencyMs) {}
}

```

2.1.2 Exécution du benchmark JMeter

Utiliser le script JMeter fourni pour générer une charge progressive

Pour chaque palier et chaque endpoint, relever :

- Le temps de réponse moyen
- Le 95e percentile
- Le débit en requêtes par seconde
- Le taux d'erreur

Comparer les courbes obtenues sur les quatre endpoints.

Identifier le palier à partir duquel l'endpoint impératif s'effondre.

Observer les logs Quarkus et le warning émis par Vert.x sur le 4ème endpoint.

Ateliers 3 : La librairie Mutiny

3.1 De *CompletableFuture* à *Mutiny*

Objectifs :

- Découvrir les types fondamentaux `Uni` et `Multi`
- Comprendre l'organisation de l'API fluente par groupes (`onItem`, `onFailure`, `onCompletion`)
- Effectuer la conversion progressive d'un code asynchrone existant

3.1.1 Mise en place du projet *mutiny-playground*

```
$ quarkus create app org.formation:mutiny-playground --no-code
$ cd mutiny-playground
$ quarkus ext add quarkus-rest-jackson
```

Récupérer la classe ***LegacyCustomerService*** fournie. Elle expose une méthode qui combine plusieurs appels asynchrones via `CompletableFuture` :

```
public CompletableFuture<CustomerProfile> loadProfile(long
customerId) {
    return findCustomerById(customerId)
        .thenCompose(customer -> findLastOrder(customer.id()))
        .thenCombine(findLoyalty(customer.id()),
            (order, loyalty) -> new CustomerProfile(customer, order,
            loyalty)))
        .exceptionally(ex -> CustomerProfile.empty(customerId));
}
```

3.1.2 Réécriture en *Mutiny*

Créer ***CustomerService*** qui expose la même API mais en retournant `Uni<CustomerProfile>`. Les trois étapes :

- Commencer par écrire la signature de la méthode
- Convertir le premier appel `findCustomerById(customerId)` en `Uni`
- À partir du `Uni<Customer>` obtenu, enchaîner les appels qui dépendent de lui. Pour cette étape, on enchaîne **séquentiellement** la commande puis la fidélité

Comparer la lisibilité des deux versions. Noter la séparation explicite des étapes par groupes (`onItem`, `onFailure`).

3.2 Composition d'appels et gestion d'erreurs

3.2.1 Séquentiel vs parallèle

Le code écrit précédemment enchaîne les trois appels séquentiellement : on attend `findCustomerById`, puis `findLastOrder`, puis `findLoyalty`.

Comme `findLastOrder` et `findLoyalty` ne dépendent pas l'un de l'autre, on peut les paralléliser.

Mesurer la version séquentielle

Ajouter dans `CustomerService` une méthode qui mesure le temps total :

```
public Uni<CustomerProfile> loadProfileSequential(long customerId)
{
    long start = System.currentTimeMillis();
    return loadProfile(customerId)
        .onItem().invoke(profile ->
            System.out.println("[sequential] " +
                (System.currentTimeMillis() - start) + " ms"));
}
```

Exposer cette méthode via un endpoint REST GET `/customers/{id}/sequential` qui retourne `Uni<CustomerProfile>`.

Appeler quelques fois avec `curl` et noter le temps moyen affiché dans la console.

Écrire la version parallèle

Découvrir le nouvel opérateur `Uni.combine().all().unis(...)` qui lance plusieurs `Uni` simultanément et combine leurs résultats :

```
public Uni<CustomerProfile> loadProfileParallel(long customerId) {
    long start = System.currentTimeMillis();
    return Uni.createFrom(
        .completionStage(() -> legacy.findCustomerById(customerId))
        .onItem().transformToUni(customer -> {
// Les 2 appels suivants ne dépendent pas l'un de l'autre.
// On les lance en parallèle avec Uni.combine().
// A compléter
        })
        .onFailure().recoverWithItem(CustomerProfile.empty(customerId))
        .onItem().invoke(profile ->
            System.out.println("[parallel] " + (System.currentTimeMillis() - start) + "
ms")));
}
```

Exposer cette méthode via GET `/customers/{id}/parallel` et appeler quelques fois.

3.2.2 Retry, timeout et fallback

Étape 1 — Simuler un service `findLoyalty` instable

Ajouter dans `LegacyCustomerService` une seconde méthode `findLoyaltyUnstable` qui

échoue de façon aléatoire et présente parfois une latence anormale :

```
private final AtomicInteger callCount = new AtomicInteger(0);

/**
 * Variante "instable" de findLoyalty : simule un service réel défaillant.
 * - 60 % des appels échouent avec une RuntimeException
 * - 20 % des appels prennent 1 seconde au lieu de 100 ms (lenteur anormale)
 * - 20 % des appels répondent normalement
 *
 * Utilisée pour démontrer retry, timeout et fallback.
 */
public CompletableFuture<Loyalty> findLoyaltyUnstable(long customerId) {
    return CompletableFuture.supplyAsync(() -> {
        int call = callCount.incrementAndGet();
        double dice = ThreadLocalRandom.current().nextDouble();

        if (dice < 0.6) {
            simulateLatency(50);
            System.out.println(" [findLoyaltyUnstable #" + call + "] échec");
            throw new RuntimeException("Service de fidélité indisponible");
        }
        if (dice < 0.8) {
            System.out.println(" [findLoyaltyUnstable #" + call + "] lenteur anormale");
            simulateLatency(1000);
        } else {
            System.out.println(" [findLoyaltyUnstable #" + call + "] succès");
            simulateLatency(100);
        }
        String tier = (customerId % 3 == 0) ? "GOLD"
            : (customerId % 2 == 0) ? "SILVER" : "BRONZE";
        return new Loyalty(customerId, tier, (int)(customerId * 137 % 5000));
    }, executor);
}
```

Étape 2 — Écrire une méthode `loadProfileResilient`

Dans `CustomerService`, ajouter une méthode qui charge un profil en utilisant la version instable de `findLoyalty`, avec une chaîne de résilience complète :

Implémenter la chaîne de résilience suivante (du haut vers le bas) :

1. Si l'appel met plus de 500 ms à répondre (cas "lenteur anormale" simulé à 1 seconde), on transforme cette absence de réponse en échec.
2. Sur tout échec (l'erreur d'origine **ou** le timeout transformé en erreur), on réessaie jusqu'à 3 fois.
3. Si après 3 tentatives c'est toujours en échec, on retombe sur une valeur par défaut construit par une méthode factory du modèle

Étape 4 — Exposer et observer

Exposer la méthode via GET `/customers/{id}/resilient` qui retourne `Uni<CustomerProfile>`.

Appeler 5 à 10 fois :

```
for i in 1 2 3 4 5; do curl -s http://localhost:8080/customers/42/resilient; echo; done
```

Bonus : retry avec back-off

Pour aller plus loin, remplacer `.atMost(3)` par un retry plus réaliste avec attente exponentielle entre chaque tentative

3.3 Concepts avancés

3.3.1 Effet latéral avec `invoke()`

Dans `CustomerService`, créer une variante de `loadProfile` qui trace chaque étape.

Exposer via GET `/customers/{id}/traced` et appeler une fois avec `curl`

3.3.2 Conversion entre `Uni` et `Multi`

Ajouter une méthode dans `CustomerService` qui charge plusieurs clients en flux :

```
public Multi<Customer> loadCustomers(int count) {  
    // On part d'un Multi qui émet les IDs 1, 2, ..., count  
  
    // Pour chaque ID, on déclenche un appel asynchrone qui renvoie un  
    // Uni<Customer>  
  
    // transformToUniAndConcatenate convertit chaque Uni en élément du flux  
    // résultant  
  
    return Multi.createFrom().range(1, count + 1)  
        .onItem().transformToUniAndConcatenate(id ->  
            Uni.createFrom().completionStage(() ->  
                legacy.findCustomerById(id)));  
}
```

Exposer un endpoint qui retourne le flux **collecté en liste**

3.3.3 Mémoriser avec `memoize()`

Ajouter dans `CustomerService` la méthode :

```
public Uni<Long> demoNoMemoize(long customerId) {  
    // Ce Uni rejoue findCustomerById à chaque souscription  
  
    Uni<Customer> customerUni = Uni.createFrom()  
        .completionStage(() -> legacy.findCustomerById(customerId));  
  
    // On s'abonne 3 fois au même Uni  
  
    Uni<Customer> sub1 = customerUni.onItem().invoke(c -> System.out.println("  
[no-memoize] sub1"));  
  
    Uni<Customer> sub2 = customerUni.onItem().invoke(c -> System.out.println("  
[no-memoize] sub2"));  
}
```

```

Uni<Customer> sub3 = customerUni.onItem().invoke(c -> System.out.println("
[no-memoize] sub3"));

return Uni.combine().all().unis(sub1, sub2, sub3)
    .with((c1, c2, c3) -> c1.id());
}

```

Exposer via GET `/customers/{id}/no-memoize` et appeler une fois. Dans la console du serveur, observer que `findCustomerById` est appelé 3 fois

Implémenter ensuite une méthode **`public Uni<Long> demoMemoize(long customerId)`** qui mémorise l'appel à `findByCustomerId`

Exposer, tester et observer le nombre d'appels à `findBuCustomerId`

3.4 Tester du code Mutiny

Ajouter la dépendance de test dans `pom.xml`, dans le bloc `<dependencies>` :

```

<dependency>
  <groupId>io.smallrye.reactive</groupId>
  <artifactId>mutiny-test-utils</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.assertj</groupId>
  <artifactId>assertj-core</artifactId>
  <version>3.27.3</version>
  <scope>test</scope>
</dependency>

```

Cette dépendance apporte `UniAssertSubscriber`, l'outil pour souscrire à un `Uni` dans un test et faire des assertions sur son comportement (item émis, erreur, complétion).

Créer le test `CustomerServiceTest` et implémenter des méthodes de test pour la méthode `loadProfile`

Ateliers 4 : Couche persistance — Hibernate Reactive et Redis

Démarrage du fil rouge : une application de gestion de commandes.

Utilisation de PostgreSQL via Panache Reactive, gérer les transactions et leurs erreurs, puis ouvrir sur la persistance NoSQL réactive avec un cache Redis simple sur les lectures de commandes.

4.1 Hibernate Reactive Panache et PostgreSQL

4.1.1 Mise en place

Sur **order-service**, ajouter les extensions :

```
$ quarkus create app org.formation:order-service --no-code
$ cd order-service
$ quarkus ext add quarkus-hibernate-reactive-panache
$ quarkus ext add quarkus-reactive-pg-client
```

Ajouter également la dépendance **lombok**

Démarrer une base PostgreSQL via le fichier **postgres-docker-compose.yml** fourni :

```
$ docker compose -f postgres-docker-compose.yml up -d
```

Se connecter à pgAdmin <http://localhost:81>

Enregistrer le serveur dans pgAdmin :

- Host : **panache-postgresql**
- User : **postgres**
- Password : **postgres**

Créer une base de données **order-service** via pgAdmin

Récupérer les classes de domaine fournies

Configurer la datasource réactive dans **application.properties** :

```
quarkus.datasource.db-kind=postgresql
quarkus.datasource.username=postgres
quarkus.datasource.password=postgres
quarkus.datasource.reactive.url=postgresql://localhost:5432/order-service
quarkus.hibernate-orm.schema-management.strategy=drop-and-create
quarkus.hibernate-orm.log.sql=true
```

Effectuer un premier démarrage et observer la création des tables

4.1.2 Repository Pattern

Créer ensuite un **OrderRepository** implémentant **PanacheRepository<Order>** et y définir deux requêtes spécifiques :

- Les commandes ayant bénéficié d'une remise (champ `discount > 0`)
- Les commandes dont la date de création est postérieure à un paramètre `LocalDateTime`

@ApplicationScoped

```
public class OrderRepository implements PanacheRepository<Order> {  
    public Uni<List<Order>> withDiscount() {  
        return list("discount > 0");  
    }  
    public Uni<List<Order>> createdAfter(Instant from) {  
        return list("createdAt > ?1", from);  
    }  
}
```

Exécuter les tests fournis de `OrderRepositoryTest`

4.1.3 Service métier réactif

Implémenter un autre bean **org.formation.service.OrderService** utilisant le repository précédent et implémenter et écrire une classe de test pour la méthode :

```
public Uni<Order> createOrder(List<OrderItem> lineItems, Address  
    deliveryAddress, PaymentInformation paymentInformation)
```

4.1.4 Transactions réactives

Créer une nouvelle entité `Event` :

```
@Data  
@Entity  
public class Event {  
  
    @Id @GeneratedValue  
    Long id;  
  
    Instant date;  
  
    String entityId;  
  
    String eventName;  
}
```

Modifier la méthode précédente afin qu'en une seule transaction, on enregistre la commande et on stocke un évènement dans la table Event.

Tracer le cycle de vie de la session Hibernate Reactive dans les logs et noter le passage du commit/rollback.

```
quarkus.log.category."org.hibernate.reactive.pool".level=TRACE
quarkus.log.category."org.hibernate.reactive.pool".min-level=TRACE
```

4.2 Cache Redis sur les lectures

Ouverture sur la persistance NoSQL réactive : Redis comme cache simple sur la lecture d'une commande, démontrant la composition de deux Uni dans un service réactif.

4.2.1 Mise en place de Redis

```
$ quarkus ext add quarkus-redis-client
```

Quarkus démarre automatiquement un container Redis en mode dev. Vérifier dans les logs la ligne Dev Services for Redis started.

4.2.2 Lecture avec cache

Injecter l'API typée **ReactiveValueCommands<String, Order>** depuis **ReactiveRedisDataSource** et composer la lecture :

```
@Inject ReactiveRedisDataSource redis;
private ValueCommands<String, Order> cache;

@PostConstruct
void init() {
    cache = redis.value(Order.class);
}

public Uni<Order> findById(long id) {
    String key = "order:" + id;
    return cache.get(key)
        .onItem().ifNull().switchTo(() ->
            orderRepository.findById(id)
                .call(o -> cache.setex(key, 60, o)));
}
```

Ecrire une classe de test qui montre l'effet du cache

Ateliers 5 : Couche web — Quarkus REST

5.1 Mise en place de endpoints REST

5.1.1 Ajout des extensions

```
$ quarkus ext add quarkus-rest-jackson
$ quarkus ext add quarkus-smallrye-openapi
$ quarkus ext add quarkus-hibernate-validator
```

5.1.2 Endpoints impératifs et réactifs

Créer la classe **OrderResource** exposant le endpoint `/orders`. Implémenter :

- Un GET `/orders` réactif retournant `Uni<List<Order>>`
- Un POST `/orders` réactif acceptant un payload `OrderRequest` et retournant `Uni<RestResponse<Void>>`
- Un GET `/orders/{id}` réactif retournant `Uni<Order>`

Le payload JSON attendu pour la création :

```
{
  "lineItems": [
    {
      "refProduct": "string",
      "price": 0.1,
      "quantity": 0
    }
  ],
  "deliveryAddress": {
    "rue": "string",
    "ville": "string",
    "codePostal": "string"
  },
  "paymentInformation": {
    "paymentToken": "7897496"
  }
}
```

Tester via Swagger

5.1.3 Bean Validation

Annoter `OrderRequest` avec les contraintes appropriées (`@NotNull`, `@NotEmpty` sur `lineItems`, `@Pattern` sur `cardNumber`) et `@Valid` dans la signature du endpoint.

Tester via Swagger les 400 lors de violation de contraintes.

5.1.4 ExceptionMapper centralisé

Avec `@ServerExceptionMapper`, retourner un 404 lorsqu'un `EntityNotFoundException` est levé

5.1.5 Tests RestAssured

Ajouter la dépendance :

```
<dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <scope>test</scope>
</dependency>
```

Écrire au minimum deux tests d'API :

- Création d'une commande valide : statut 201 et id dans l'entête en réponse
- Création d'une commande sans lineItems : statut 400 et message de validation

@QuarkusTest

```
class OrderResourceTest {
    @Test
    void createOrder_valid_returns201() {
        given().contentType(ContentType.JSON).body(validPayload())
            .when().post("/orders")
            .then().statusCode(201);
    }
}
```

Ateliers 6 : Couche service — composition et résilience

Enrichir la création de commande par des appels en parallèle à deux services externes (catalogue et stock), avec une politique de résilience face à des services lents ou indisponibles.

6.1 Quarkus REST Client

Objectifs :

- Déclarer des clients HTTP réactifs via l'extension **quarkus-rest-client**
- Composer en parallèle des appels à plusieurs services
- Mettre en œuvre **@Retry**, **@Timeout**, **@CircuitBreaker**, **@Fallback**

6.1.1 Démarrage des services dépendants

Démarrer les 2 projets quarkus et visitez leur DevUI

6.1.2 Déclaration des clients

Sur le projet **order-service** :

```
$ quarkus ext add quarkus-rest-client-jackson
$ quarkus ext add quarkus-smallrye-fault-tolerance
```

Déclarer deux interfaces de client : **CatalogClient** et **StockClient**. Chacune expose une méthode réactive :

```
@RegisterRestClient(configKey = "catalog-api")
public interface CatalogClient {
    @GET @Path("/products/{ref}")
    Uni<Product> findById(@PathParam("ref") String ref);
}
```

Configurer les URLs racines dans **application.properties** :

```
quarkus.rest-client.catalog-api.url=http://localhost:9001
quarkus.rest-client.stock-api.url=http://localhost:9002
```

6.1.2 Composition en parallèle

Dans le service métier lors du `findById`, enrichir chaque `OrderItem` par les appels parallèles aux deux clients :

```
private Uni<EnrichedItem> enrichItem(OrderItem item) {
    Uni<Product> productUni =
catalogClient.findById(item.productId())
        .onSubscription().invoke(() -> Log.infof("→ [%d] CATALOG", item.productId()));

    Uni<Integer> stockUni =
stockClient.available(item.productId())
```

```

        .onSubscription().invoke(() -> Log.infof("→ [%d] STOCK", item.productId()));

    return Uni.combine().all()
        .unis(productUni, stockUni)
        .asTuple()
        .map(t -> new EnrichedItem(item, t.getItem1(), t.getItem2()));
}

```

Vérifier dans les logs que les deux appels partent bien simultanément.

6.2 Politiques de résilience

6.2.1 Retry et Timeout

Sur `CatalogClient.findById`, ajouter les annotations de tolérance aux pannes :

```

@GET @Path("/products/{id}")
@Timeout(800)
@Retry(maxRetries = 2, delay = 200)
Uni<Product> findById(@PathParam("id") long id);

```

6.2.2 Circuit Breaker et Fallback

Au niveau du service métier, encadrer l'appel par un `@CircuitBreaker` et fournir une méthode de fallback retournant un produit « inconnu » :

```

@CircuitBreaker(requestVolumeThreshold = 4, failureRatio = 0.5,
    delay = 5000)
@Fallback(fallbackMethod = "unknownProduct")
public Uni<Product> safeFindProduct(long id) {
    return catalogClient.findById(id);
}

Uni<Product> unknownProduct(long id) {
    return Uni.createFrom().item(Product.unknown(id));
}

```

6.2.3 Mise à l'épreuve

Simuler une panne du service catalogue (l'arrêter) et observer :

- Les retries déclenchés dans les logs du client

- L'ouverture du circuit breaker après plusieurs échecs successifs
- Le fallback retourné instantanément quand le circuit est ouvert

Ateliers 7 : SmallRye Reactive Messaging et Kafka

Brancher la couche messaging du fil rouge — publication d'événements Kafka à la création de commande, consommation par un service de notification — puis instrumenter le tout avec OpenTelemetry pour observer la propagation du tracing à travers les couches.

7.1 Émission de messages

Sur *order-service*, ajouter les extensions :

```
$ quarkus ext add quarkus-messaging-kafka
```

Quarkus démarre automatiquement un container Kafka en mode dev. Accéder à la Dev UI et vérifier l'onglet Kafka.

Récupérer la classe de domaine *OrderEvent* fournie et créer un service *PublishService* qui émet un événement à chaque création :

```
@ApplicationScoped
public class PublishService {
    @Channel("orders-out")
    MutinyEmitter<Event> emitter;

    public Uni<Void> publish(Event event) {
        return emitter.send(event);
    }
}
```

Configurer le channel dans *application.properties* (le topic Kafka cible est nommé orders) :

```
mp.messaging.outgoing.orders-out.connector=smallrye-kafka
mp.messaging.outgoing.orders-out.topic=orders-event
```

Intégrer *PublishService* dans le flux de création de commande. Vérifier dans la Dev UI Kafka que les messages arrivent bien.

7.2 Consommation côté notification-service

Créer un nouveau projet *notification-service* :

```
$ quarkus create app org.formation:notification-service --no-code
$ cd notification-service
$ quarkus ext add quarkus-messaging-kafka
$ quarkus ext add quarkus-rest-jackson
```

Récupérer la classe *OrderEvent* (identique à celle d'*order-service*). Écrire un consumer :

```
@ApplicationScoped
```

```

public class OrderConsumer {
    @Incoming("orders-in")
    public Uni<Void> consume(OrderEvent event) {
        return switch (event.type()) {
            case CREATED -> notifyCustomerCreated(event);
            case CANCELLED -> notifyCustomerCancelled(event);
        };
    }
}

mp.messaging.incoming.orders-in.connector=smallrye-kafka
mp.messaging.incoming.orders-in.topic=orders
mp.messaging.incoming.orders-in.failure-strategy=dead-letter-queue

```

Démarrer les deux services en parallèle. Créer une commande dans order-service via Swagger et vérifier la trace de notification dans la console de notification-service.

7.3 SSE

Dans le projet *order-service*

- Ajouter si besoin l'extension quarkus-rest-jackson
- Créer une ressource GET sur le endpoint /orders/stream qui renvoie un

Multi<Event> correspondant au topic orders au format MediaType.SERVER_SENT_EVENTS

- Récupérer le fichier orders.html et le placer dans src/main/resources/META-INF/resources
- Le comprendre